4.4 ARMAX Method: Aggregate-Daily (Store Type A)

2026-04-08

Goal

Build and compare ARMAX models for aggregate-daily Store Type A sales.

This notebook:

1. Reads aggregate processed files directly.
2. Uses a single train/holdout split.
3. Selects the best ARMAX specification by train AIC.
4. Evaluates the selected model on holdout using fixed-coefficient 1-step forecasts.
5. Produces aggregate-daily test forecasts.

Libraries and Data

```r
library(tidyverse)
library(forecast)

seasonal_frequency <- 7L

train_data <- readr::read_csv(
  "../../data/processed/store_type_a_train_aggregate_daily.csv",
  show_col_types = FALSE
) |>
  dplyr::mutate(
    Date = as.Date(Date),
    PromoShare = dplyr::if_else(OpenStores > 0, PromoStores / OpenStores, 0),
    DayOfWeek = as.integer(DayOfWeek),
    Month = as.integer(Month)
  ) |>
  dplyr::arrange(Date)

test_data <- readr::read_csv(
  "../../data/processed/store_type_a_test_aggregate_daily.csv",
  show_col_types = FALSE
) |>
  dplyr::mutate(
    Date = as.Date(Date),
    PromoShare = dplyr::if_else(OpenStores > 0, PromoStores / OpenStores, 0),
    DayOfWeek = as.integer(DayOfWeek),
    Month = as.integer(Month)
  ) |>
  dplyr::arrange(Date)

head(train_data)
```

```
## # A tibble: 6 x 11
```

```
##   Date         Sales OpenStores PromoStores PromoShare SchoolHolidayShare
##   <date>       <dbl>      <dbl>       <dbl>      <dbl>              <dbl>
## 1 2013-01-01    2907          1           0          0             1
## 2 2013-01-02 3920386        602           0          0             1
## 3 2013-01-03 3528027        601           0          0             0.939
## 4 2013-01-04 3610215        601           0          0             0.939
## 5 2013-01-05 2964587        600           0          0             0.0664
## 6 2013-01-06    6254          3           0          0             0.0664
## # i 5 more variables: DayOfWeek <int>, Year <dbl>, Month <int>, IsoWeek <dbl>,
## #   IsWeekend <dbl>
```

Forecast Information Set Assumption

This ARMAX setup is a conditional forecast: we assume day-t inputs are known when forecasting day t.

For a 1-step-ahead forecast at day t, the model uses:

$$\mathcal{F}_{t-1} \cup \{X_t^{planned}\}$$

Here, X_t includes `PromoShare`, `OpenStores`, and calendar covariates.

Practical interpretation:

1. `DayOfWeek` and `Month` are deterministic and known in advance.
2. `PromoShare` and `OpenStores` are valid only if planned and available by the end of day $t-1$.
3. If you use realized day-t values, holdout RMSE may be overly optimistic due to information leakage beyond $\mathcal{F}_{t-1}$.

Candidate ARMAX Specifications

```r
model_specs <- list(
  ARMAX_Promo_Open = stats::as.formula("~ PromoShare + OpenStores"),
  ARMAX_Promo_Open_DOW = stats::as.formula("~ PromoShare + OpenStores + DayOfWeek"),
  ARMAX_Promo_Open_DOW_Month = stats::as.formula("~ PromoShare + OpenStores + DayOfWeek + Month")
)

model_specs
```

```
## $ARMAX_Promo_Open
## ~PromoShare + OpenStores
##
## $ARMAX_Promo_Open_DOW
## ~PromoShare + OpenStores + DayOfWeek
##
## $ARMAX_Promo_Open_DOW_Month
## ~PromoShare + OpenStores + DayOfWeek + Month
```

Holdout Split Setup

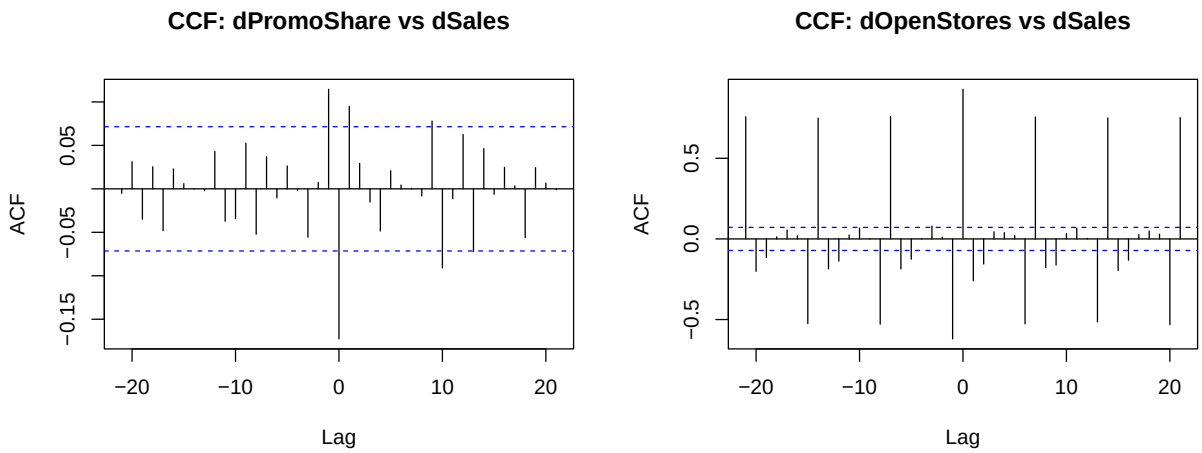
```r
set.seed(443)

holdout_prop <- 0.20
n_total <- nrow(train_data)
n_train <- floor((1 - holdout_prop) * n_total)
```

```r
train_subset <- train_data |> dplyr::slice(1:n_train)
holdout_subset <- train_data |> dplyr::slice((n_train + 1):n_total)

holdout_split <- tibble::tibble(
  n_total = n_total,
  n_train = n_train,
  n_holdout = n_total - n_train,
  train_start = min(train_subset$Date),
  train_end = max(train_subset$Date),
  holdout_start = min(holdout_subset$Date),
  holdout_end = max(holdout_subset$Date)
)

holdout_split
```

```
## # A tibble: 1 x 7
##   n_total n_train n_holdout train_start train_end  holdout_start holdout_end
##     <int>   <dbl>     <dbl> <date>      <date>     <date>        <date>
## 1     942     753       189 2013-01-01  2015-01-23 2015-01-24    2015-07-31
```

CCF on Differenced Series

```r
diag_data <- train_subset |>
  dplyr::transmute(
    Sales,
    PromoShare,
    OpenStores
  ) |>
  dplyr::mutate(
    dSales = c(NA_real_, diff(Sales)),
    dPromoShare = c(NA_real_, diff(PromoShare)),
    dOpenStores = c(NA_real_, diff(OpenStores))
  ) |>
  tidyr::drop_na()

old_par <- par(no.readonly = TRUE)
par(mfrow = c(1, 2))

stats::ccf(
  diag_data$dPromoShare,
  diag_data$dSales,
  lag.max = 21,
  main = "CCF: dPromoShare vs dSales"
)

stats::ccf(
  diag_data$dOpenStores,
  diag_data$dSales,
  lag.max = 21,
  main = "CCF: dOpenStores vs dSales"
)
```




```
par(old_par)
```

ARMAX Utilities

```
build_xreg <- function(df, model_formula) {
  df <- df |>
    dplyr::mutate(
      DayOfWeek = factor(DayOfWeek, levels = 1:7),
      Month = factor(Month, levels = 1:12)
    )

  mm <- stats::model.matrix(model_formula, data = df)
  mm[, colnames(mm) != "(Intercept)", drop = FALSE]
}

rmse_vec <- function(actual, pred) {
  sqrt(mean((actual - pred) ^ 2, na.rm = TRUE))
}

clean_xreg <- function(xreg_train, xreg_valid) {
  xreg_train <- as.matrix(xreg_train)
  xreg_valid <- as.matrix(xreg_valid)

  keep_non_constant <- apply(xreg_train, 2, function(x) stats::var(x, na.rm = TRUE) > 0)
  xreg_train <- xreg_train[, keep_non_constant, drop = FALSE]
  xreg_valid <- xreg_valid[, keep_non_constant, drop = FALSE]

  qr_fit <- qr(xreg_train)
  keep_idx <- sort(qr_fit$pivot[seq_len(qr_fit$rank)])
  xreg_train <- xreg_train[, keep_idx, drop = FALSE]
  xreg_valid <- xreg_valid[, keep_idx, drop = FALSE]

  list(train = xreg_train, valid = xreg_valid)
}

prepare_xreg_for_split <- function(data, model_formula, n_train) {
  xreg_full_raw <- build_xreg(data, model_formula)
```

```r
  xreg_train_raw <- xreg_full_raw[seq_len(n_train), , drop = FALSE]

  xreg_clean <- clean_xreg(xreg_train_raw, xreg_full_raw)

  list(
    train = xreg_clean$train,
    full = xreg_clean$valid
  )
}

fit_armax_train <- function(data, model_formula, n_train) {
  y_train <- ts(data$Sales[seq_len(n_train)], frequency = seasonal_frequency)
  xreg_obj <- prepare_xreg_for_split(data, model_formula, n_train)

  fit <- forecast::auto.arima(
    y_train,
    xreg = xreg_obj$train,
    seasonal = TRUE,
    allowmean = FALSE,
    allowdrift = FALSE,
    stepwise = TRUE,
    approximation = TRUE
  )

  list(
    fit = fit,
    xreg_full = xreg_obj$full
  )
}

resolve_seasonal_period <- function(ord, y_ts, fallback_period = 7L) {
  m_raw <- suppressWarnings(as.integer(unname(ord["m"])))
  freq_raw <- suppressWarnings(as.integer(stats::frequency(y_ts)))

  if (!is.na(m_raw) && m_raw > 1L) {
    return(m_raw)
  }

  if (!is.na(freq_raw) && freq_raw > 1L) {
    return(freq_raw)
  }

  fallback_period
}

extract_arima_spec <- function(train_fit_obj, y_ts) {
  ord <- forecast::arimaorder(train_fit_obj)

  list(
    ord = ord,
    order = c(
      as.integer(unname(ord["p"])),
      as.integer(unname(ord["d"])),
```

```r
      as.integer(unname(ord["q"]))
    ),
    seasonal_order = c(
      as.integer(unname(ord["P"])),
      as.integer(unname(ord["D"])),
      as.integer(unname(ord["Q"]))
    ),
    seasonal_period_raw = unname(ord["m"]),
    seasonal_period_used = resolve_seasonal_period(ord, y_ts)
  )
}

fixed_coef_holdout_forecast <- function(data, n_train, train_fit_obj, xreg_full) {
  y_all <- ts(data$Sales, frequency = seasonal_frequency)
  spec <- extract_arima_spec(train_fit_obj, y_all)

  fixed_fit <- stats::arima(
    y_all,
    order = spec$order,
    seasonal = list(
      order = spec$seasonal_order,
      period = spec$seasonal_period_used
    ),
    xreg = xreg_full,
    include.mean = FALSE,
    fixed = as.numeric(train_fit_obj$coef),
    transform.pars = FALSE,
    optim.control = list(maxit = 0),
    method = "ML"
  )

  fc_all <- as.numeric(y_all - fixed_fit$residuals)
  holdout_idx <- (n_train + 1):length(y_all)

  tibble::tibble(
    Date = data$Date[holdout_idx],
    actual = as.numeric(y_all[holdout_idx]),
    predicted = fc_all[holdout_idx]
  )
}
```

Select ARMAX Specification by Train AIC

```r
y_train_for_period <- ts(train_data$Sales[seq_len(n_train)], frequency = seasonal_frequency)

train_fit_registry <- purrr::imap(model_specs, function(model_formula, model_name) {
  fit_armax_train(
    data = train_data,
    model_formula = model_formula,
    n_train = n_train
  )
})
```

```r
aic_table <- purrr::imap_dfr(train_fit_registry, function(obj, model_name) {
  spec <- extract_arima_spec(obj$fit, y_train_for_period)
  seasonal_period_label <- as.character(as.integer(spec$seasonal_period_used))

  tibble::tibble(
    model = model_name,
    train_aic = stats::AIC(obj$fit),
    seasonal_period_raw = spec$seasonal_period_raw,
    seasonal_period_used = spec$seasonal_period_used,
    arima_order = sprintf(
      "(%d,%d,%d)(%d,%d,%d)[%s]",
      spec$order[1],
      spec$order[2],
      spec$order[3],
      spec$seasonal_order[1],
      spec$seasonal_order[2],
      spec$seasonal_order[3],
      seasonal_period_label
    )
  )
}) |>
  dplyr::arrange(train_aic)

aic_table
```

```
## # A tibble: 3 x 5
##   model           train_aic seasonal_period_raw seasonal_period_used arima_order
##   <chr>               <dbl>               <int>                <int> <chr>
## 1 ARMAX_Promo_Op~    21906.                  NA                    7 (2,0,2)(2,~
## 2 ARMAX_Promo_Op~    21921.                  NA                    7 (2,0,5)(2,~
## 3 ARMAX_Promo_Op~    22073.                  NA                    7 (2,0,2)(2,~
```

```r
seasonal_period_issues <- aic_table |>
  dplyr::filter(is.na(seasonal_period_raw))

if (nrow(seasonal_period_issues) > 0) {
  warning("At least one ARMAX candidate reports NA seasonal period from arimaorder(). Using seasonal_pe
}

seasonal_period_issues
```

```
## # A tibble: 3 x 5
##   model           train_aic seasonal_period_raw seasonal_period_used arima_order
##   <chr>               <dbl>               <int>                <int> <chr>
## 1 ARMAX_Promo_Op~    21906.                  NA                    7 (2,0,2)(2,~
## 2 ARMAX_Promo_Op~    21921.                  NA                    7 (2,0,5)(2,~
## 3 ARMAX_Promo_Op~    22073.                  NA                    7 (2,0,2)(2,~
```

Fixed-Coefficient 1-Step Holdout Evaluation

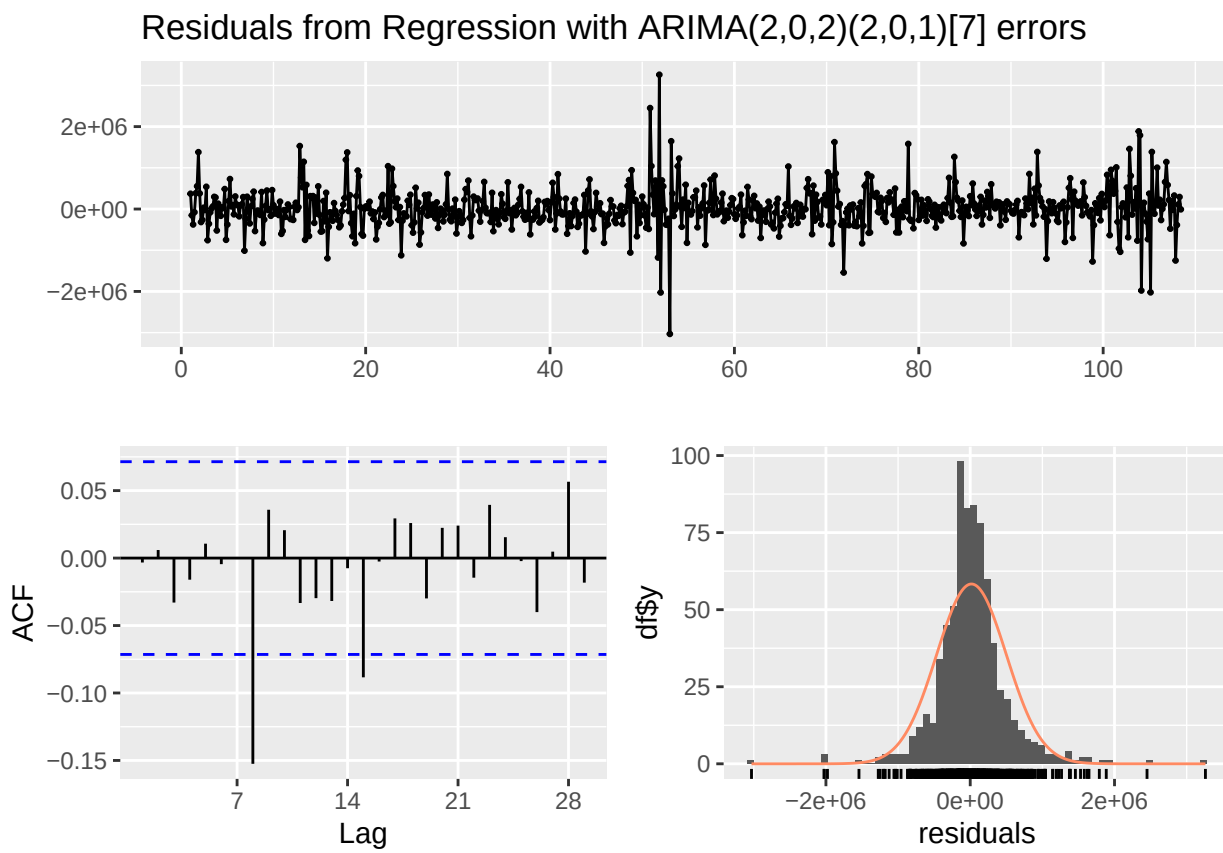
```r
best_model <- aic_table |>
  dplyr::slice_min(order_by = train_aic, n = 1)

best_model_name <- best_model$model[[1]]
```

```r
best_model_formula <- model_specs[[best_model_name]]
best_train_fit <- train_fit_registry[[best_model_name]]$fit
best_xreg_full <- train_fit_registry[[best_model_name]]$xreg_full

holdout_predictions <- fixed_coef_holdout_forecast(
  data = train_data,
  n_train = n_train,
  train_fit_obj = best_train_fit,
  xreg_full = best_xreg_full
)

armax_holdout_summary <- tibble::tibble(
  model = best_model_name,
  train_aic = best_model$train_aic[[1]],
  holdout_rmse = rmse_vec(holdout_predictions$actual, holdout_predictions$predicted),
  n_holdout = nrow(holdout_predictions)
)

armax_holdout_summary
```

```
## # A tibble: 1 x 4
##   model                      train_aic holdout_rmse n_holdout
##   <chr>                          <dbl>        <dbl>     <int>
## 1 ARMAX_Promo_Open_DOW_Month    21906.      399948.       189
```

Residual Diagnostics on Selected Train Fit

```r
forecast::checkresiduals(best_train_fit)
```

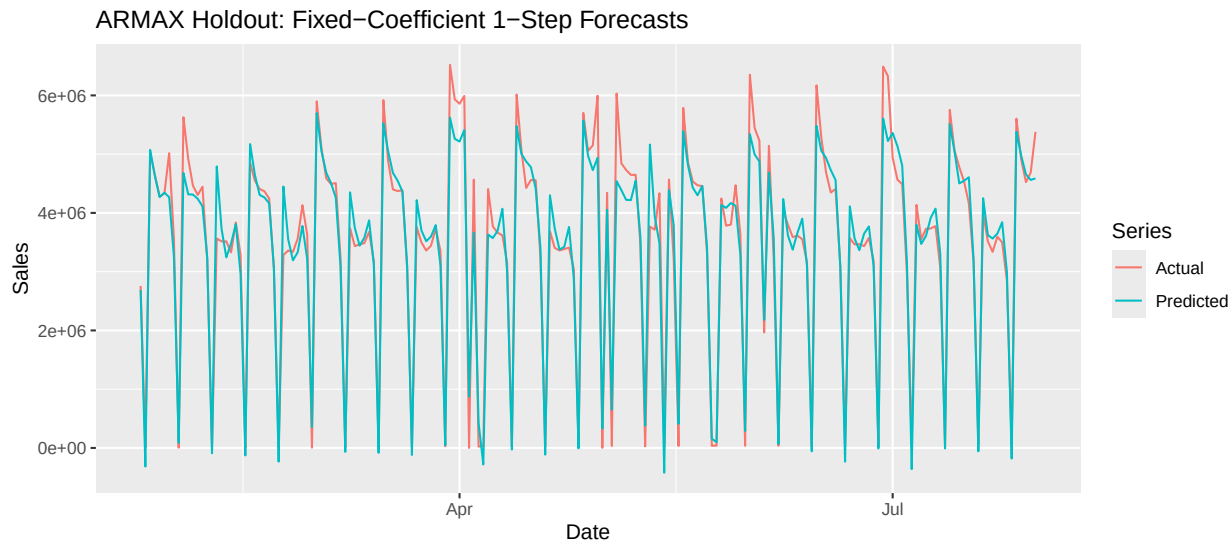



```
##
##  Ljung-Box test
##
## data:  Residuals from Regression with ARIMA(2,0,2)(2,0,1)[7] errors
## Q* = 22.552, df = 7, p-value = 0.002039
##
## Model df: 7.   Total lags used: 14
```

```
residual_diagnostics <- tibble::tibble(
  model = best_model_name,
  sigma2 = best_train_fit$sigma2,
  ljung_box_pvalue = stats::Box.test(
    residuals(best_train_fit),
    lag = 14,
    type = "Ljung-Box",
    fitdf = sum(best_train_fit$arma[1:4])
  )$p.value
)

residual_diagnostics
```

```
## # A tibble: 1 x 3
##   model                              sigma2 ljung_box_pvalue
##   <chr>                               <dbl>            <dbl>
## 1 ARMAX_Promo_Open_DOW_Month 242257433257.          0.00204
```

```r
ggplot(holdout_predictions, aes(x = Date)) +
  geom_line(aes(y = actual, color = "Actual")) +
  geom_line(aes(y = predicted, color = "Predicted")) +
  labs(
    title = "ARMAX Holdout: Fixed-Coefficient 1-Step Forecasts",
    x = "Date",
    y = "Sales",
    color = "Series"
  )
```



Best ARMAX Model

```r
best_model
```

```
## # A tibble: 1 x 5
##   model           train_aic seasonal_period_raw seasonal_period_used arima_order
##   <chr>               <dbl>               <int>                 <int> <chr>
## 1 ARMAX_Promo_Op~    21906.                  NA                     7 (2,0,2)(2,~
```

```r
best_model_name
```

```
## [1] "ARMAX_Promo_Open_DOW_Month"
```

```r
best_model_formula
```

```
## ~PromoShare + OpenStores + DayOfWeek + Month
```

Fit Best Model on Full Train and Forecast Test Horizon

```r
y_full <- ts(train_data$Sales, frequency = seasonal_frequency)

xreg_train_full_raw <- build_xreg(train_data, best_model_formula)
xreg_test_raw <- build_xreg(test_data, best_model_formula)

xreg_clean_full <- clean_xreg(xreg_train_full_raw, xreg_test_raw)
xreg_train_full <- xreg_clean_full$train
xreg_test <- xreg_clean_full$valid
```

```r
best_fit <- forecast::auto.arima(
  y_full,
  xreg = xreg_train_full,
  seasonal = TRUE,
  allowmean = FALSE,
  allowdrift = FALSE,
  stepwise = TRUE,
  approximation = TRUE
)
test_forecast <- as.numeric(forecast::forecast(best_fit, xreg = xreg_test, h = nrow(test_data))$mean)

aggregate_test_forecast <- test_data |>
  dplyr::transmute(
    Date,
    predicted_aggregate_sales = pmax(0, test_forecast)
  )

head(aggregate_test_forecast)
```

```
## # A tibble: 6 x 2
##   Date       predicted_aggregate_sales
##   <date>                         <dbl>
## 1 2015-08-01                  2515834.
## 2 2015-08-02                   288768.
## 3 2015-08-03                  3487401.
## 4 2015-08-04                  2847204.
## 5 2015-08-05                  2554873.
## 6 2015-08-06                  2526256.
```

Save Outputs

```r
dir.create("results", showWarnings = FALSE, recursive = TRUE)

readr::write_csv(holdout_split, "results/armax_holdout_split.csv")
readr::write_csv(aic_table, "results/armax_aic_table.csv")
readr::write_csv(holdout_predictions, "results/armax_holdout_predictions.csv")
readr::write_csv(armax_holdout_summary, "results/armax_holdout_rmse_summary.csv")
readr::write_csv(aggregate_test_forecast, "results/armax_aggregate_test_forecast.csv")
```